COMPUTER ORGANIZATION AND ARCHITECTURE (21CSS201T)

Unit 3 : Tutorial 1 — De Morgan's Theorem & Design/Analysis of Basic and Advanced Adders
Name: JATANPREET SINGH VIRDI JATANPREET SINGH VIRDI | Reg. No: RA2511003011374 | Solved Worksheet

Part 1: De Morgan's Theorem

• Theorem 1: The complement of a sum of variables equals the product of their complements: (A + B)' = A' . B'

• Theorem 2: The complement of a product of variables equals the sum of their complements: (A . B)' = A' + B'

• Equivalent circuits: Theorem 1 — a NOR gate equals an AND gate with inverted inputs (bubbled AND). Theorem 2 — a NAND gate equals an OR gate with inverted inputs (bubbled OR).

• Simplifications: a) (A + B)' = A' . B' b) (X . Y)' = X' + Y' c) (A' + B' + C')' = A . B . C d) ((A + B) . C)' = (A + B)' + C' = A'B' + C'

• Proof of Theorem 1 by truth table: rows (A,B) = 00/01/10/11 give (A+B)' = 1,0,0,0 and A'.B' = 1,0,0,0 — identical columns, hence proved.

• Chemical process alarm: let H = heater, F = fan, P = pump. Process active = H.F.P; alarm X = (H.F.P)'. By De Morgan: X = H' + F' + P' — the alarm sounds if the heater OR fan OR pump is off; implement with an OR gate with inverted inputs.

Part 2: Design and Analysis of Basic and Advanced Adders

• Half adder truth table: (A,B) 00→S0 C0; 01→S1 C0; 10→S1 C0; 11→S0 C1. Sum = A'B + AB' = A ⊕ B; Carry = A . B. Logic diagram: one XOR gate for Sum, one AND gate for Carry.

• Full adder truth table: the 8 rows give S = A ⊕ B ⊕ Cin and Cout = 1 whenever two or more inputs are 1. SOP forms: Sum = A'B'Cin + A'BCin' + AB'Cin' + ABCin; Cout = A'BCin + AB'Cin + ABCin' + ABCin = AB + BCin + ACin.

• Full adder from two half adders + OR gate: HA1 adds A and B; HA2 adds HA1's sum with Cin; the two carry outputs go to an OR gate producing Cout.

• 4-bit ripple-carry adder: four full adders cascaded, the carry-out of each feeding the carry-in of the next higher stage.

• Main disadvantage: propagation delay — each stage waits for the previous carry, so the delay ripples from LSB to MSB. With 10 ns per full adder, total delay = 4 × 10 ns = 40 ns.

• Adding A = 1101 and B = 1011: bit0 1+1+0 → S0=0 c=1; bit1 0+1+1 → S1=0 c=1; bit2 1+0+1 → S2=0 c=1; bit3 1+1+1 → S3=1 c=1. Sum = 10000 (5-bit result, 13 + 11 = 24), showing the ripple effect.

• Carry-lookahead adder: computes the carry of every stage directly from the inputs instead of waiting for the ripple. Advantage: much higher speed because all carries are generated in parallel.

• Generate Gi = Ai . Bi (carry created when both inputs are 1). Propagate Pi = Ai ⊕ Bi (carry passed on when exactly one input is 1).

• Comparison — Ripple-carry: slow (delay ∝ number of bits), small area, low complexity. Carry-lookahead: fast (delay grows logarithmically), larger area, high complexity.

• A = 1011, B = 0110 ripple addition: Bit0: A0=1, B0=0, Cin=0 → S0=1, c=0; Bit1: A1=1, B1=1, Cin=0 → S1=0, c=1; Bit2: A2=0, B2=1, Cin=1 → S2=0, c=1; Bit3: A3=1, B3=0, Cin=1 → S3=0, c=1. Final Sum = 0001, Carry-out = 1.